E7: Introduction to Computer Programming
Midterm 1A

Statement of Academic Integrity:

"As a member of the UC Berkeley community, I act with honesty, integrity, and respect for others."

Write your name to confirm that all work on this exam will be your own. Do not discuss contents of the exam even after the exam ends, as some students may be taking the exam at a different time. The penalty for academic misconduct on an exam is an F in the course. All incidents will be reported to the Center for Student Conduct.

Write your name above AND on the next page.

Do not open the exam until instructed to do so.

You may remove this sheet and use it for scratch work.

You still have to turn this sheet in with your exam.

E7: Introduction to Computer Programming
Midterm 1A

Tuesday, September 29

Full Name

Student ID Number

Instructions:

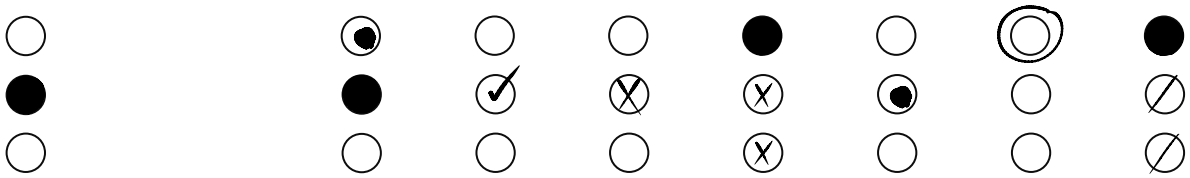
- You have 1 hour and 20 minutes to complete this exam.
- The exam is closed book, closed notes, closed computer, closed calculator.
- You may only use one 8.5" × 11" sheet of notes (both sides). No other resources are permitted.
- Mark your answers on the exam paper within the designated spaces. Responses written on scratch paper or outside of the specified answer spaces will not be graded.
- This exam is DOUBLE-SIDED. Make sure to read both sides of each page.
- The number of points associated with each part of each question is indicated.
- The exam is worth a total of 100 points.
- For questions with circular bubbles, you should select exactly *one* choice.
  - ◯ You may only select one option!
  - ◯ Or this one, but not both!
  - ◯ Or this one, but no more than one!
- For questions with square boxes, you may select *multiple* choices.
  - ☐ You can select this one!
  - ☐ You can select this one too!
  - ☐ You can select one, two, or all three of these!
- Only include your FINAL answers in the designated boxes. Any additional text will count as part of your answer and may result in a reduced grade.
- In all questions, `Error` means that an error occurs when the code is executed.

Good luck!

Question 1 (30 points)

There are 10 multiple choice questions. There is only **ONE** correct choice for each question. Multiple bubbles, incomplete bubbles, or stray marks will be marked incorrect.

Correct Method: Wrong Methods:



1. (3 pt) What is the value of `y` after executing the following code?

```
x = [4, 7, 2, 9]
y = x[1] + x[-1]
```

○ 9 ○ 11 ○ [7, 9] ○ Error ○ 13 ● 16

2. (3 pt) What will be displayed after executing the following code?

```
def fun(x):
    x = x - 1
    return x

x = 5
y = fun(x)
print(x)
```

○ 6 ○ 1 ○ 4 ● 5 ○ Error ○ -1

3. (3 pt) Which expression correctly checks whether `x` is between 0 and 10, inclusive?

● `x >= 0 and x <= 10`
○ `x >= 0 or x <= 10`
○ `0 < x < 10`
○ `0 > x > 10`
○ `x > 0 and x < 10`
○ `x > 0 or x < 10`

4. (3 pt) What is the purpose of the `if` statement in the function below?

```
def fun(n):
    if n == 1:
        return 1
    return n * fun(n-2)
```

○ It defines the parameter
○ It changes the value of `n`
○ It makes the function recursive
○ It repeats the function
● It stops the recursion
○ It sets a default value for `n`

5. (3 pt) What is the final value of `total` after executing the following code?

```
for a in [1, 3, 6]:
    total = 0
    total += a
```

- (●) 6
- ( ) [1, 3, 6, 0]
- ( ) 0
- ( ) 10
- ( ) 3
- ( ) Error
- ( ) 1
- ( ) 4

6. (3 pt) Which of the following is a built-in Python data structure that is mutable?

- ( ) ndarray
- ( ) tuple
- (●) list
- ( ) int
- ( ) range
- ( ) str

7. (3 pt) What is the value of `answer` after executing the following code?

```
def study():
    print("I know this.")
answer = study()
```

- ( ) str
- (●) None
- ( ) study()
- ( ) 0
- ( ) Error
- ( ) "I know this."

8. (3 pt) How many times is the value of `a` printed when the following code runs?

```
for a in range(4):
    print(a)
print(a)
```

- ( ) 6
- ( ) 8
- ( ) 2
- ( ) 3
- ( ) 4
- (●) 5
- ( ) Error
- ( ) 0

9. (3 pt) Assume that `x = 4`, `y = 7`, and `L = [2, 4, 6]`. Which of the following expressions evaluates to `False`?

- ( ) `not (x > y or y in L)`
- ( ) `x < y and x in L`
- ( ) `x == y - 3`
- ( ) `x != y`
- (●) `x not in L and y > x`
- ( ) `x == 4 or y == 4 and x > y`

10. (3 pt) Assume that `V1 = [1, 1, 1]` and `V2 = [2, 3, 5]` have been defined and NumPy has been imported as `np`. Which of the following produces a different sequence of numbers?

- ( ) `[3, 4] + [6]`
- ( ) `np.array(V1) + np.array(V2)`
- (●) `np.array(V1 + V2)`
- ( ) `np.array([1, 2, 4]) + 2`
- ( ) None of the options
- ( ) `3, 4, 6`

Question 2 (13 points)

For parts (a), (b), and (c), the parameter list of each function has been removed. You are given example function calls and their outputs. Use this information to determine which parameter list(s) could reproduce all given outputs for each function. Assume no global variables exist.

(a) (3 pt) Which parameter list could replace blank (a)? Select ONE answer.

```
def mystery1(___(a)___):
    """
    >>> mystery1(10)
    2.0
    >>> mystery1()
    Error
    """
    return x / y
```

- ○ y, x=5
- ○ x, y
- ○ x=1, y=2
- ○ x, y=2
- ● x, y=5
- ○ x
- ○ y, x

(b) (4 pt) Which parameter lists could replace blank (b)? Select ALL that apply.

```
def mystery2(___(b)___):
    """
    >>> mystery2(2, 10)
    8
    """
    return x - y
```

- ☐ a, b
- ■ y, x
- ☐ Y, X
- ☐ X, Y
- ☐ x, y
- ■ y, x=2

(c) (2 pt) Which parameter list could replace blank (c)? Select ONE answer.

```
def mystery3(___(c)___):
    """
    >>> mystery3(10)
    5
    >>> mystery3(10, 2)
    25
    """
    return x + y * z
```

- ○ x, y=10, z=2
- ○ x, y, z
- ○ y, x=5, z=2
- ○ y, z, x=5
- ○ x, y, z=2.5
- ● y, z=0, x=5
- ○ x, y=0, z=1

(d) (4 pt) Which of the following function calls would cause an `Error`? Select ALL that apply.

```
def mystery4(x, y, z=3, w=4):
    # function body omitted
```

- ☐ mystery4(0, 1, 4, 0)
- ■ mystery4(2)
- ☐ mystery4(1, 2)
- ☐ mystery4(1, 4, w=2)
- ■ mystery4(1, 4, 3, 0, 0)
- ☐ mystery4(w=1, y=4, z=2, x=0)
- ☐ mystery4(1, 4, 3)
- ■ mystery4(x=1, 4, 3, 0)

Question 3 (10 points)

Assume the function below has been defined. Note that `list.append(x)` appends item `x` to the end of the list.

```
def f(sequence, target, flag):
    output = [ ]
    for x in sequence:
        if x < 0:
            continue
        if x == target:
            output.append(x * 10)
        elif x % 2 == 0:
            output.append(x)
        if flag and x > 1:
            output.append(99)
        if x == 4:
            break
    return output
```

For all parts, what is returned by each of the following function calls? If an error occurs, write Error. Only include your FINAL answers in the designated boxes. Any text will count as part of your answer and may result in a reduced grade. You can use the space outside the boxes for scratch.

(a) (4 pt) `f([1, 2, 3, 4, 2], 2, False)`

`[20, 4]`

(b) (3 pt) `f([1, 2, 3, 4, 2], 2, True)`

`[20, 99, 99, 4, 99]`

(c) (3 pt) `f([2, -2, 2], -2, True)`

`[2, 99, 2, 99]`

Question 4 (12 points)

Assume the code below has been executed.

```
def first(a):
    global x
    x += a
    b = second(x - 2)
    return b + a

def second(y):
    y = y * 2
    x = 3
    return y + x

x = 6
y = 1
```

For all parts, what is the output of the following expressions? If an error occurs, write Error. Only include your FINAL answers in the designated boxes. Any text will count as part of your answer and may result in a reduced grade. You can use the space outside the boxes for scratch.

(a) (4 pt) `print(second(y))`

5

(b) (3 pt) `print(second(x))`

15

For parts (c) and (d), assume the code snippet below is executed in the same environment as the previous code, in the order shown.

```
z = first(y)
w = second(x)
```

(c) (3 pt) `print(z)`

14

(d) (2 pt) `print(w)`

17

Question 5 (13 points)

(10 pt) For parts (i) – (iv), determine the smallest integer value of `x` for which the specified output will be displayed. Enter your FINAL answers in the designated boxes.

(i)
```
if x > 5:
    if x > 10:
        print('a')
    elif x > 7:
        print('b')
    else:
        print('c')
else:
    print('d')
```

Output: c

x = 6

(iii)
```
if x < 10:
    print('a')
elif x < 15:
    print('b')
elif x < 20:
    print('c')
else:
    print('d')
```

Output: d

x = 20

(ii)
```
if x > 5:
    print('a')
elif x > 0:
    print('b')
if x != 6:
    print('c')
else:
    print('d')
```

Output: a
c

x = 7

(iv)
```
for i in range(x):
    for j in range(3):
        if i == 1:
            break
        if j != 1:
            continue
        print('a')
```

Output: a
a

x = 3

(3 pt) What will be displayed after executing the following code? The output may have multiple lines. If an error occurs, write Error. If nothing is displayed, write Nothing. If this runs forever, write Forever.

(v)
```
seq = [0, 1, 4, 1, 2]
for i, val in enumerate(seq):
    if i == 0 or i == len(seq) - 1:
        continue
    if i > seq[val]:
        print('A')
    elif i == val:
        print('B')
```

Output:

B
A

Question 6 (18 points)

You need to write a Python function `single_digit` that takes an integer `n` and repeatedly calculates the sum of its digits until the result is a single digit. Assume `n` is a nonnegative integer (you don't need to check its value).

Your function should return an integer representing the final single-digit result. Refer to the examples below for more details.

- For `n = 9992`, the output is `2` because $9+9+9+2 = 29$, then $2+9 = 11$, and finally $1+1 = 2$.
- For `n = 1234`, the output is `1` because $1 + 2 + 3 + 4 = 10$, then $1 + 0 = 1$.

There are different ways to implement this function. The first approach below is based on converting `n` to a string.

```
def single_digit(n):
    """Repeatedly sum digits of n until a single digit remains.
    >>> single_digit(9992)
    2
    >>> single_digit(1234)
    1
    >>> single_digit(7) # already a single digit
    7
    """
    ______(a)______:
        digit_sum = 0
        nstr = str(n)
        for ______(b)______:
            digit_sum += ______(c)______
        ______(d)______ = digit_sum
______(e)______ # return omitted. See part (e) for indentation options.
```

The parts for this question begin on the next page. You can use this page for scratch work. Anything written on this page will NOT be graded. Only answers within the designated boxes on the next page will be graded.

You will implement the function `single_digit` by filling in the blanks. You may only write one statement on each line. You may not use ; to place two statements on the same line. You may not modify the level of indentation of any line.

For parts (a) – (d), you may only use built-in functions that we discussed in lecture. No other functions or methods are allowed.

(a) (2 pt) Fill in blank (a). You must use built-in function(s) discussed in lecture. Answers that do not use built-in function(s) as a fundamental part of the solution will not receive credit.

```
while len(str(n)) > 1
```

(b) (2 pt) Which option correctly fills blank (b)?

○ `digit = nstr` ○ `digit in str` ● `digit in nstr`
○ `digit in range(nstr)` ○ `digit in n` ○ `digit in range(n)`

(c) (2 pt) Fill in blank (c).

```
digit_sum += int(digit)
```

(d) (1 pt) Fill in blank (d).

```
n = digit_sum
```

(e) (3 pt) Which option correctly fills blank (e)?

○
```
def single_digit(n):
    __(a)__:
        digit_sum = 0
        nstr = str(n)
        for __(b)__:
            digit_sum += __(c)__
        __(d)__ = digit_sum
        return n
```

●
```
def single_digit(n):
    __(a)__:
        digit_sum = 0
        nstr = str(n)
        for __(b)__:
            digit_sum += __(c)__
        __(d)__ = digit_sum
    return n
```

○
```
def single_digit(n):
    __(a)__:
        digit_sum = 0
        nstr = str(n)
        for __(b)__:
            digit_sum += __(c)__
        __(d)__ = digit_sum
            return n
```

○
```
def single_digit(n):
    __(a)__:
        digit_sum = 0
        nstr = str(n)
        for __(b)__:
            digit_sum += __(c)__
        __(d)__ = digit_sum
return n
```

Next, you will implement the same function but using a different approach. For parts (f) – (i), you may NOT use any functions or methods, built-in or otherwise.

```
def single_digit(n):
    # same test cases as before
    ______(f)______:
        digit_sum = 0
        ______(g)______:
            digit_sum += ______(h)______
            n = ______(i)______
        ______(d)______ = digit_sum # same answer as blank (d)
______(e)______ # same answer as blank (e)
```

(f) (2 pt) Fill in blank (f).

```
    while n >= 10
```

(g) (2 pt) Fill in blank (g).

```
    while n > 0
```

(h) (2 pt) Fill in blank (h).

```
digit_sum += n % 10
```

(i) (2 pt) Fill in blank (i).

```
n = n // 10
```

Question 7 (4 points)

A happy number is a number that eventually reaches `1` when the squares of its digits are repeatedly summed. If the process reaches `4`, the number is not happy. For example, 19 is a happy number:

$$19 \to 1^2 + 9^2 = 82 \to 8^2 + 2^2 = 68 \to 6^2 + 8^2 = 100 \to 1^2 + 0^2 + 0^2 = 1$$

However, 145 is an unhappy number:

$$145 \to 1^2 + 4^2 + 5^2 = 42 \to 4^2 + 2^2 = 20 \to 2^2 + 0^2 = 4$$

You need to write a Python function `happy` that takes an integer `n` and returns `True` if `n` is a happy number, `False` otherwise. Assume `n` is a nonnegative integer (you don't need to check its value).

```
def happy(n):
    """Repeatedly sum square digits of n until it is a happy or unhappy number.
    >>> happy(19)
    True
    >>> happy(10)
    True
    >>> happy(145)
    False
    >>> happy(20)
    False
    """
    if ______(a)______:
        return True
    ______(b)______:
        return False
    return ______(c)______
```

The parts for this question begin on the next page. You can use this page for scratch work. Anything written on this page will NOT be graded. Only answers within the designated boxes on the next page will be graded.

You will implement the function `happy` by filling in the blanks. You may only write one statement on each line. You may not use ; to place two statements on the same line. You may not modify the level of indentation of any line. You may only use built-in functions that we discussed in lecture in addition to the `happy` function itself. No other functions or methods are allowed. The built-in `sum()` function may be helpful.

(a) (1 pt) Fill in blank (a).

```
if n == 1
```

(b) (1 pt) Fill in blank (b).

```
    if n == 4
```

(c) (2 pt) Fill in blank (c). No partial credit will be awarded for incorrect solutions.

```
return happy(sum(int(i) ** 2 for i in str(n)))
```